

Ejercicio: Implementación Robusta del Método de Bisección

Objetivo: Implementar en C el algoritmo numérico de bisección para encontrar ceros de funciones continuas.

Fundamento Teórico

El método de bisección se basa en el **Teorema de Bolzano**: si una función continua $f(x)$ cambia de signo en un intervalo cerrado $[a, b]$ (es decir, $f(a) \cdot f(b) \leq 0$), existe al menos un valor $c \in [a, b]$ tal que $f(c) = 0$.

El algoritmo divide repetidamente el intervalo a la mitad:

1. Se calcula el punto medio $c = \frac{a+b}{2}$.
2. Si $f(c) = 0$ o la semiamplitud del intervalo $\frac{b-a}{2} < tol$, se ha encontrado la raíz aproximada.
3. Si $f(a) \cdot f(c) < 0$, la raíz está en $[a, c]$, por lo que el nuevo intervalo es $[a, c]$ ($b = c$).
4. En caso contrario, la raíz está en $[c, b]$, por lo que el nuevo intervalo es $[c, b]$ ($a = c$).

Consigna de Programación

Diseñar e implementar en C una función modular basada en el siguiente contrato e interfaz:

```
int biseccion(float a, float b, float error, float* raiz);
```

- a y b: son los dos extremos del intervalo.
- raiz: es la raíz calculada.
- error: es la tolerancia de la solución. El programa termina cuando alcan
- retorno: la función retorna un código según
 - 0: si el método terminó bien y calculó la raíz.
 - 1: si no se cumple la condición del intervalo.
 - 2: iteró muchas veces pero no llegó a calcular la raíz con el error p

Casos de prueba

- el método debe llamar a una función `float funcion1(float x)` que iremos comentando y descomentando (más adelante veremos mejores formas de hacer esto).

Ejecutar los siguientes casos:

1. Funcion $y = x^2 - 4$;, intervalo=[0.0, 3.0], error=1e-5
2. Funcion $y = x^3 - x - 2$;, intervalo=[1.0, 2.0], error=1e-6
3. Funcion $y = x - 5$;, intervalo=[5.0, 10.0], error=1e-4
4. Funcion $y = x^2 + 1$, intervalo=[-1.0, 1.0], error=1e-4